

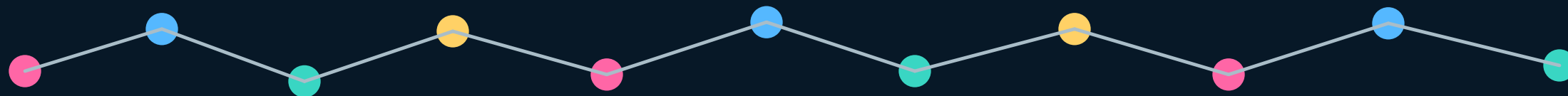
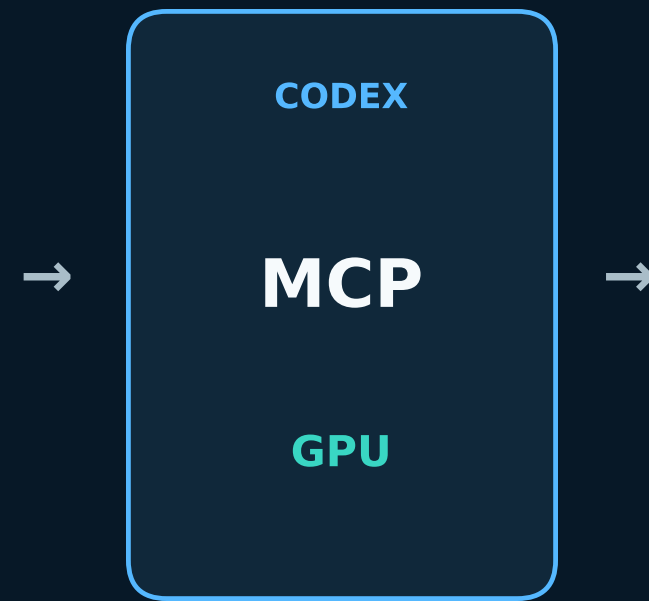
GPU MCP

Let Codex operate the GPU workflow

Safe hands for prolonged polymer-simulation investigation

NOT JUST JOB SUBMISSION

AN ITERATIVE SCIENCE LOOP



MCP in one sentence

MCP gives Codex a typed toolbelt connected to real systems.



MODEL CONTEXT PROTOCOL

Think “USB/API standard for AI tools” — not another model, and not magic autonomy.

MCP is the connection — the goal, reasoning, and scientific judgment remain separate.

The workflow friction we are removing

TODAY: COMMAND-AND-REMEMBER

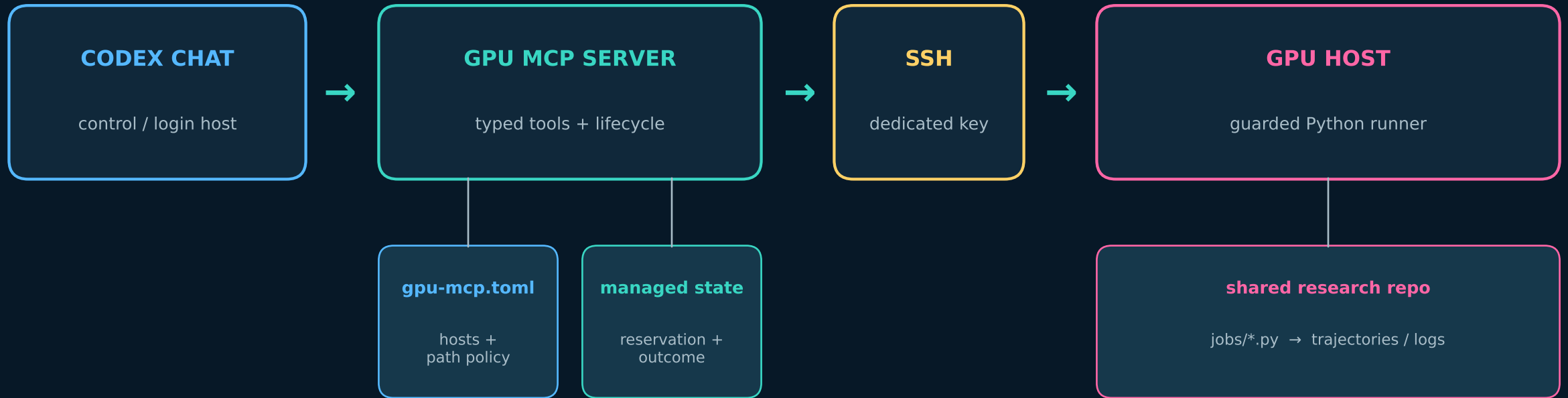
- 1 Check nvidia-smi on several hosts
- 2 Pick a GPU from stale snapshots
- 3 SSH / nohup / redirect logs
- 4 Remember host, PID and output path
- 5 Poll, interpret exit, clean up

WITH GPU MCP: INVESTIGATE-AND-ITERATE

- 1 Define a goal + stopping condition
- 2 Inspect code, inputs, GPUs, outputs
- 3 Run bounded smoke / main experiments
- 4 Analyze evidence; revise the setup
- 5 Repeat until criterion or real blocker

POLYMER-SIMULATION PAYOFF Not just less shepherding: a sustained investigate → run → learn loop on our machines.

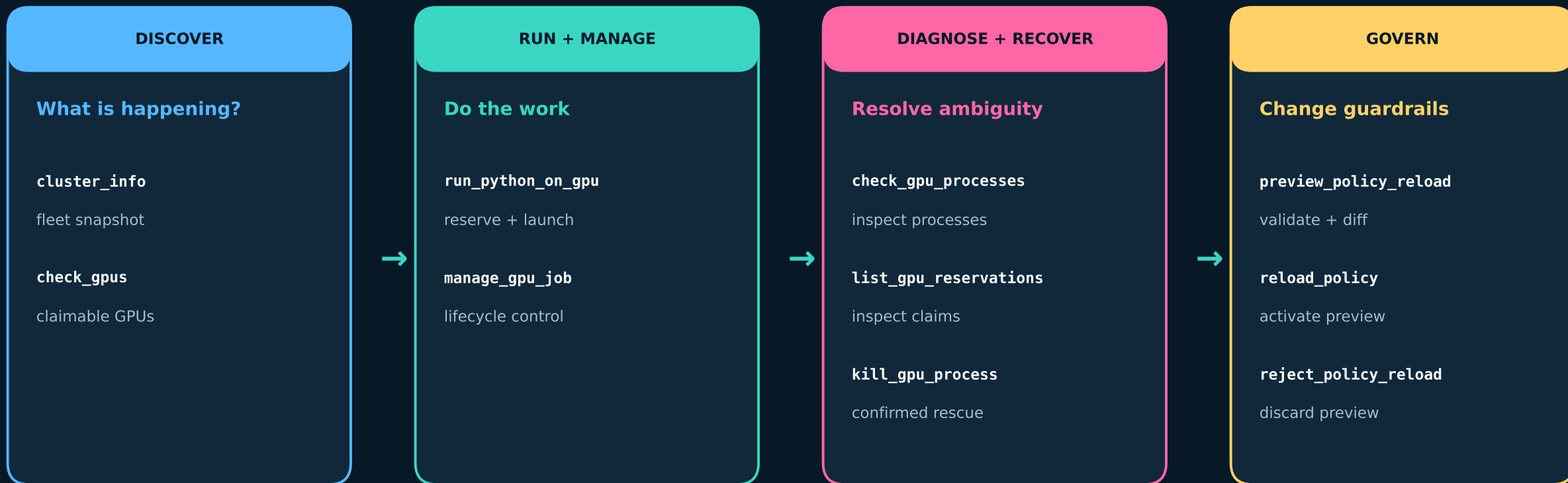
What happens after you ask Codex



MCP gives Codex safe machine-facing hands; the repo remains our scientific workspace.

The MCP is an ensemble of tools

No single tool is “the MCP” — the value comes from how the tools compose.



The tools are the hands: observe → claim → run → verify → intervene → govern

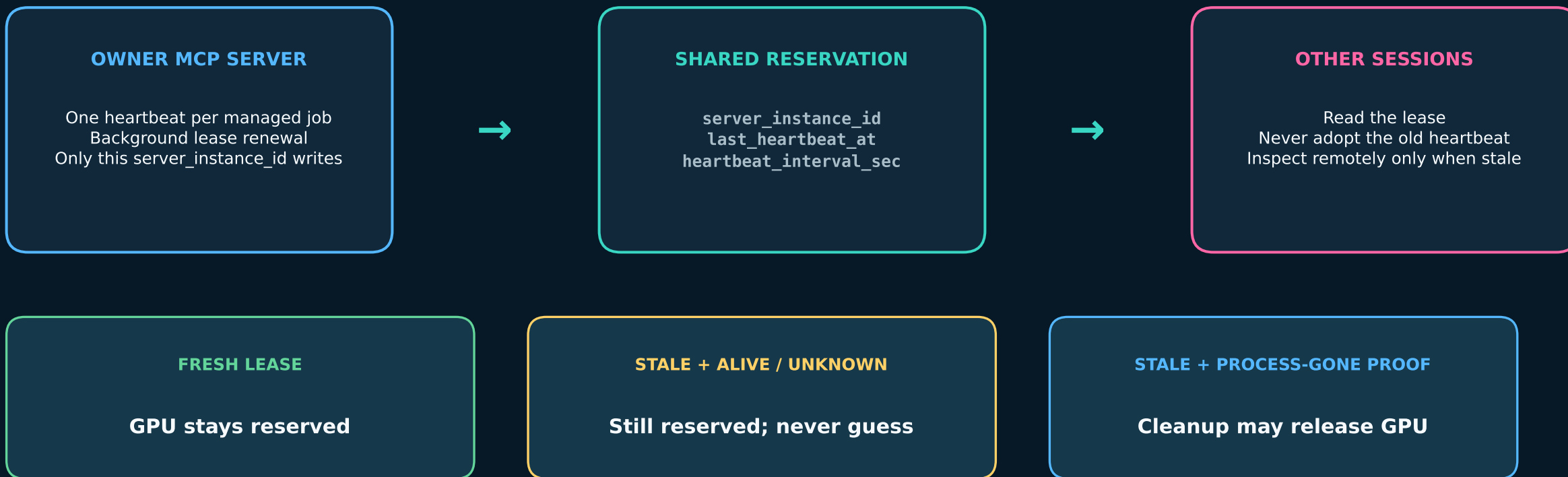
The everyday run loop: three tools



The `job_id` connects one experiment's launch to the evidence for the next decision.

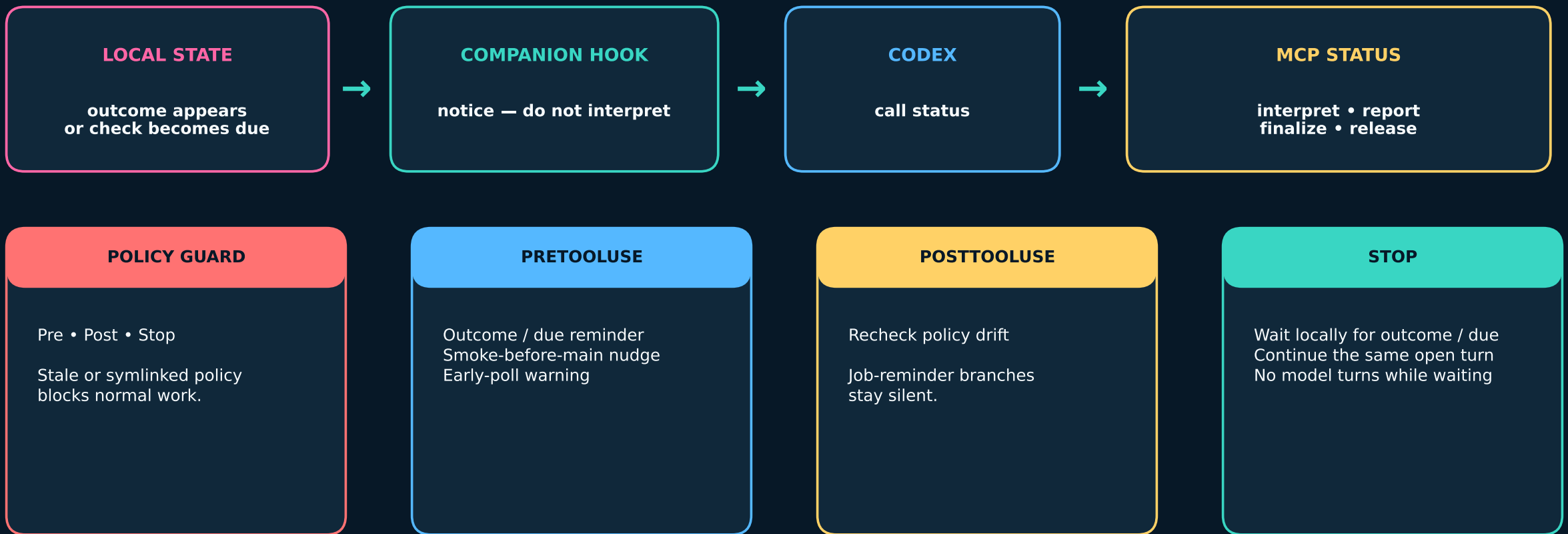
Heartbeat = ownership lease, not process liveness

It says “this MCP server still owns the reservation” — not “the Python process is alive.”



A missed heartbeat triggers inspection. It never proves that the GPU is free.

Hooks are Codex's attention layer



Hooks never SSH, parse outcomes, heartbeat, or release a GPU — status remains the authority.

The supporting tools: see, recover, govern

SEE THE FLEET

`cluster_info`

Reachability, GPU counts, average utilization, load.

`check_gpu_processes`

Host / GPU / PID / user / command and memory for compute processes.

RECOVER SAFELY

`list_gpu_reservations`

Active claims for me or everyone; optional bounded refresh.

`kill_gpu_process`

Two-step, fingerprint-confirmed rescue for one owned PID.

GOVERN POLICY

`preview_policy_reload`

Validate the changed policy, show the diff, issue a one-time token.

`reload_policy`

Activate that exact approved hash.

`reject_policy_reload`

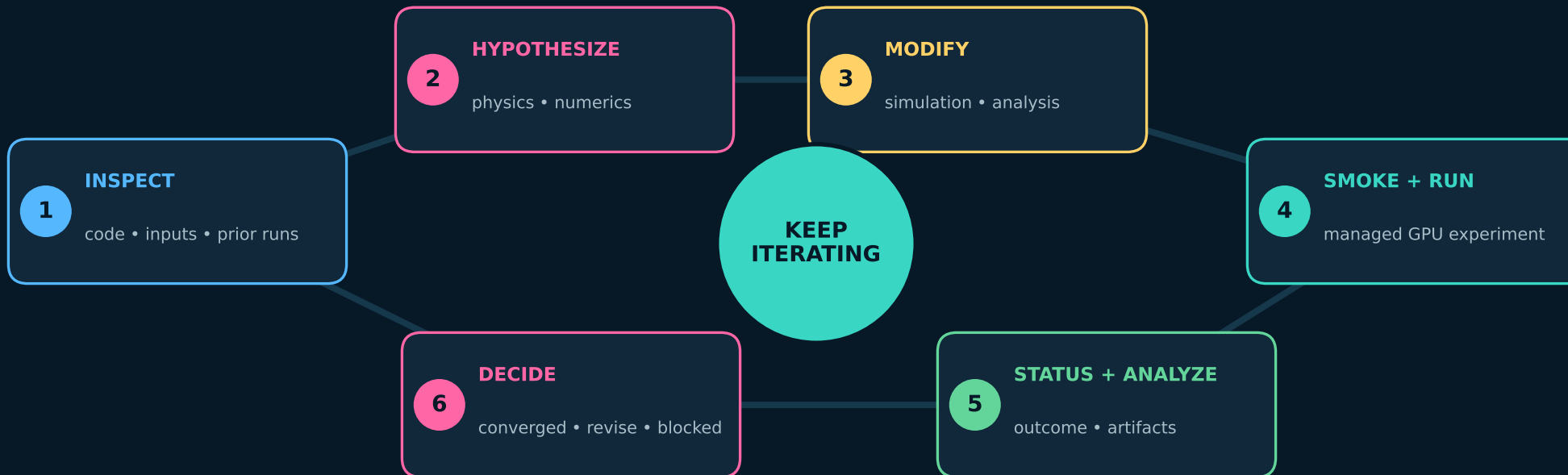
Discard the token; change nothing.

These tools make the normal run loop observable, recoverable, and policy-bounded.

The real goal: sustained scientific investigation

/GOAL

Explain why this bead-spring melt misses [criterion]. Inspect, run bounded tests, revise, and stop only at evidence or a genuine blocker.



/goal keeps the objective • heartbeat protects the run • Stop returns Codex to the next decision

Bounded autonomy: the safety model

1

REPO POLICY

approved hosts and script / write roots

2

GUARDED PYTHON

static scan + runtime audit hook; bounded writes

3

GPU RESERVATION

atomic claim prevents cooperative double-booking

4

MANAGED JOB

heartbeat, outcome record, explicit job_id

5

FAIL CLOSED

unknown process state stays reserved

WHAT IT IS

**A cooperative guardrail
for trusted lab workflows**

**A safe action surface
for sustained iteration**

WHAT IT IS NOT

**Not Slurm or a queue
Not a hostile-code sandbox
Not a validator of force fields
or equilibration**

Three things to remember

1

Hands, not just visibility

Codex can act through typed tools;
the server keeps authority.

2

A loop, not a launch

Goal + tools + heartbeat + hooks
keep investigation moving.

3

Evidence, not magic

We define credible physics
and the stopping condition.

GOOD FIRST GOAL

**Give Codex one simulation question, a bounded experiment path,
and a verifiable stopping condition.**

Questions?